Security Audit - Predicate

conducted by Neodyme AG

Lead Auditor:	Nico Gründel
Second Auditor:	Jasper Slusallek
Administrative Lead:	Jasper Slusallek

January 29, 2026



Table of Contents

1 Executive Summary **3**

2 Introduction **4**
Summary of Findings 4

3 Scope **5**

4 Project Overview **6**
Functionality 6
On-Chain Data and Accounts 6
Instructions 7
Authority Structure and Off-Chain Components 7

5 Findings **8**
[ND-PRED-CR-01] Replay protection can be circumvented during the clock drift grace period . 9
[ND-PRED-LO-01] client_program on PolicyAccount can be set to arbitrary values 10
[ND-PRED-LO-02] client_program on PolicyAccount can be set to arbitrary values 11
[ND-PRED-LO-03] Hash collision in Statements is possible 12
[ND-PRED-IN-01] client_program isn't verified in UpdatePolicyId 13
[ND-PRED-IN-02] No domain separation for signed message 14

Appendices

A About Neodyme **15**

B Methodology **16**
Select Common Vulnerabilities 16

C Vulnerability Severity Rating **18**

1 | Executive Summary

Neodyme audited **Predicate Labs's** on-chain Predicate program from November 2025 until January 2026.

According to Neodyme's Rating Classification, **4 security-relevant** and **2 informational** were found. The number of findings identified throughout the audit, grouped by severity, can be seen in Figure 1.

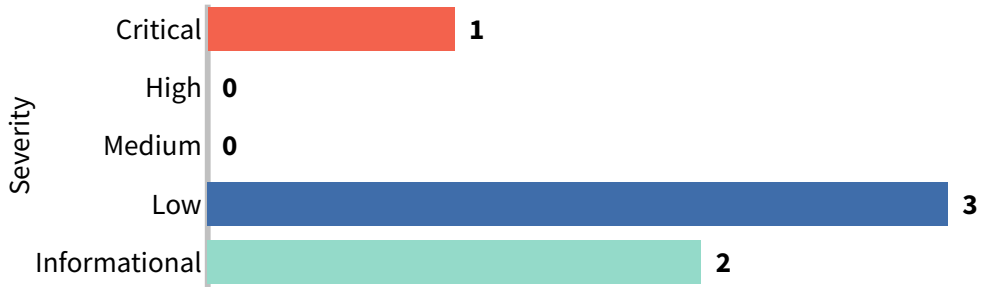


Figure 1: Overview of Findings

The auditors reported all findings to the Predicate Labs developers, who addressed them promptly. The security fixes were verified for completeness by Neodyme. In addition to these findings, Neodyme delivered the Predicate Labs team a list of nit-picks and additional notes that are not part of this report.

2 | Introduction

From November 2025 until January 2026, Predicate Labs engaged Neodyme to do a detailed security analysis of their on-chain contract. Two senior security researchers from Neodyme conducted independent full audits of the contract between the 18th of November 2025 and the 29th of January 2026. Both auditors have a long track record of finding critical and other vulnerabilities in Solana programs, as well as in Solana's core code itself.

The audit focused on the contract's technical security. In the following sections, we present our findings and discuss worst-case scenarios for authority compromise, and provide some general notes for considerations that may be useful in the future.

Predicate Labs's team always responded quickly and competently to findings of any kind. The contract's source code has no unnecessary dependencies, relying mainly on the well-established Anchor framework.

Summary of Findings

All found issues were **quickly remediated**. In total, the audit revealed:

1 critical • **0** high-severity • **0** medium-severity • **3** low-severity • **2** informational

issues. The issues found are detailed in Section 5.

3 | Scope

The contract audit's scope comprised of two major components:

- Primarily, the **Implementation** security of the contract's source code
- Additionally, security of the **overall design**

Neodyme considers the source code, located at https://github.com/predicatelabs/sol-contracts/tree/main/programs, in scope for this audit. Third-party dependencies are not in scope. Predicate Labs only relies on the Anchor library, the spl-token program and their Rust crate `hex`, all of which are well-established. During the audit, minor changes and fixes were made by Predicate Labs, which the auditors also reviewed in-depth.

Relevant source code revisions are:

- `c0f73619edb402cafdc2ddcdb8332d2b0d94aeb8` · Start of the audit
- `34af4ea32f5483c68de5ef34e8c009f52ce11155` · Last reviewed revision

4 | Project Overview

This section briefly outlines Predicate's functionality, design, and architecture, followed by a detailed discussion of all related authorities.

Functionality

Predicate provides on-chain infrastructure for other smart contracts: it maintains a set of Attesters and a set of Policies. Attesters can sign Statements for Policies, that then get validated on-chain by the Predicate smart contract.

A `Statement` has the following structure:

```rust
/// Unique identifier for the statement
pub uuid: [u8; 16],
/// The message sender
pub msg_sender: Pubkey,
/// The target address
pub target: Pubkey,
/// The message value (in lamports for Solana)
pub msg_value: u64,
/// Encoded signature and arguments
pub encoded_sig_and_args: Vec<u8>,
/// The policy ID (string identifier, not content)
pub policy_id: String,
/// Expiration timestamp
pub expiration: i64,
```

On-Chain Data and Accounts

The `Registry` is a global singleton for this program. It contains a single global authority that can register and deregister Attesters as well as transfer its own authority.

`PolicyAccount` accounts are managed by upgrade authorities of programs. Their address is derived from the programs address, making it possible for programs to create `PolicyAccount` accounts and later on verify `Statements` referencing them. `Statements` always reference a `policy_id` string, which can be set and updated in the `PolicyAccount` account by the programs upgrade authority.

`Attesters` are public keys that can sign `Statements`.

For each verified `Statement` attestation, a UUID account is created whose address is derived from the `Statement`. This prevents `Attestation` replays. They can be cleaned up after the `Statement` is expired and can no longer be verified.

Instructions

The contract has a total of 8 instructions, which we briefly summarize here.

Instruction	Category	Summary
Initialize	Admin-Only	Initializes the global Registry singleton
TransferAuthority	Admin-Only	Transfers the authority over the global Registry singleton to a different account
RegisterAttester	Admin-Only	Adds an Attester to the attester-set
DeregisterAttester	Admin-Only	Removes an Attester from the attester-set
SetPolicyId	Program-Update-Authority-Only	Creates a new PolicyId account with a specified policy_id
UpdatePolicyId	Program-Update-Authority-Only	Updates the policy_id of a PolicyId
ValidateAttestation	MsgSender-Only	Checks that an active Attester signed a statement, and creates a UUID account to prevent replay protection
CleanupExpiredUUID	Permissionless	Cleans up a UUID account belonging to an expired Statement, and returns the Rent to the account that originally paid for it

Authority Structure and Off-Chain Components

There are three types of authorities in Predicates system: the Registry-authority, the program's update authority, and Attesters:

- The Registry-authority is responsible for maintaining the set of Attesters.
- Attesters are off-chain signers that generate the signatures for Statements that are validated in ValidateAttestation.
- The update authority belonging to UpgradableLoader programs can create and manage their own PolicyAccount.

In addition, statements contain a `msg_sender` that is responsible for signing the ValidateAttestation instruction for validating the Statement.

5 | Findings

This section outlines all of our findings. They are classified into one of five severity levels, detailed in Appendix C. In addition to these findings, Neodyme delivered the Predicate Labs team a list of nit-picks and additional notes which are not part of this report.

All findings are listed in Table 2 and further described in the following sections.

Identifier	Name	Severity	Status
ND-PRED-CR-01	Replay protection can be circumvented during the clock drift grace period	CRITICAL	Fixed
ND-PRED-LO-01	client_program on PolicyAccount can be set to arbitrary values	LOW	Fixed
ND-PRED-LO-02	client_program on PolicyAccount can be set to arbitrary values	LOW	Fixed
ND-PRED-LO-03	Hash collision in Statements is possible	LOW	Fixed
ND-PRED-IN-01	client_program isn't verified in UpdatePolicyId	INFORMATIONAL	Fixed
ND-PRED-IN-02	No domain separation for signed message	INFORMATIONAL	Fixed

Table 2: Findings

[ND-PRED-CR-01] Replay protection can be circumvented during the clock drift grace period

Severity	Impact	Affected Component	Status
CRITICAL	Unlimited message replay	Attestation Verification	Fixed

The ValidateAttestation instruction allows for attestations to be valid for up to 30s after the attestation has expired. However, the CleanupExpiredUuid instruction allows for the Uuid account to be deleted as soon as the attestation has expired.

This means one can validate an attestation and delete the Uuid over and over again from the expiry time of the attestation up to 30s afterwards. In the Counter example contract, this would mean that one could increment the counter an arbitrary number of times (up to transaction count limits per block) with just one attestation.

Relevant Code

```
// Check if statement has expired (with small buffer for clock drift)
const CLOCK_DRIFT_BUFFER: i64 = 30; // 30 seconds buffer
require!(
    current_timestamp <= statement.expiration + CLOCK_DRIFT_BUFFER,
    PredicateRegistryError::StatementExpired
);
```

Mitigation Suggestion

Either take clock drift into account in both instructions, or in neither.

Resolution

Fixed in https://github.com/predicatelabs/sol-contracts/pull/15.

[ND-PRED-LO-01] client_program on PolicyAccount can be set to arbitrary values

Severity	Impact	Affected Component	Status
LOW	Limited scope and priviledged DoS, invalid chain state	Attester Registration	Fixed

SetPolicyId has a parameter `client_program` that isn't verified in any way, but is then used to initialize the `policy_id` field in the `PolicyAccount`, and additionally in the emision of the `PolicySet` event.

This will result in a corrupt `PolicyAccount` that can't be used in `VerifyAttestation`, as here the `client_program` inside the `PolicyAccount` is used to derive its address.

Depending on how the bot reads and reacts to `PolicyAccount` accounts and events, this might also result in further, more critical issues.

Mitigation Suggestion

Remove the parameter entirely, and instead use the pubkey of the corresponding account.

Resolution

Fixed in https://github.com/predicatelabs/sol-contracts/pull/16

[ND-PRED-LO-02] client_program on PolicyAccount can be set to arbitrary values

Severity	Impact	Affected Component	Status
LOW	Limited scope and priviledged DoS	Attester Registration	Fixed

When an attester is deregistered, the corresponding account does not get deleted. Furthermore, there's an init constraint on the attester account in RegisterAttester. This means the account can't be initialized again, and hence that an attester can't be registered again after they have been deregistered.

This seems to be unintentional. Note also that it is a behavioral mismatch to the Ethereum contract, where re-registering attesters is allowed: https://github.com/predicatelabs/predicate-contracts/blob/2c8a12649d926ca2b216bb097324c8c6e8fd51bc/src/PredicateRegistry.sol#L61-L68

Relevant Code

```
/// Hash the statement for signature verification (equivalent to
hashStatementSafe in Solidity)
pub fn hash_statement_safe(&self) -> [u8; 32] {
    use anchor_lang::solana_program::hash::hash;

    let mut data = Vec::new();
    data.extend_from_slice(&self.uuid);
    data.extend_from_slice(&self.msg_sender.to_bytes());
    data.extend_from_slice(&self.target.to_bytes());
    data.extend_from_slice(&self.msg_value.to_le_bytes());
    data.extend_from_slice(&self.encoded_sig_and_args);
    data.extend_from_slice(self.policy_id.as_bytes());
    data.extend_from_slice(&self.expiration.to_le_bytes());

    hash(&data).to_bytes()
}
```

Mitigation Suggestion

Delete the attester account when deregistering them, or alternatively make the registering of attesters use an init_if_needed constraint.

Resolution

Fixed in https://github.com/predicatelabs/sol-contracts/pull/17

[ND-PRED-LO-03] Hash collision in Statements is possible

Severity	Impact	Affected Component	Status
LOW	Potential footgun for devs	Attestation	Fixed

In ValidateAttestation, the function hash_statement_safe is used. This first serializes the statement parameters in an unstructured byte vector, and then hashes that vector. Since both encoded_sig_and_args and policy_id are variable-length, it is possible to have a hash collision between separate inputs when policy_id and encoded_sig_and_args are crafted just right.

As one example, if the policy_id is changed to something that is the suffix of the old policy_id, it may be possible for an attacker to use an old attestation that has not expired yet and to control encoded_sig_and_args in a way that results in the same bytestring getting hashed, and therefore the same hash. This might not be clear to devs using this contract.

Relevant Code

```rust
/// Hash the statement for signature verification (equivalent to
hashStatementSafe in Solidity)
pub fn hash_statement_safe(&self) -> [u8; 32] {
    use anchor_lang::solana_program::hash::hash;

    let mut data = Vec::new();
    data.extend_from_slice(&self.uuid);
    data.extend_from_slice(&self.msg_sender.to_bytes());
    data.extend_from_slice(&self.target.to_bytes());
    data.extend_from_slice(&self.msg_value.to_le_bytes());
    data.extend_from_slice(&self.encoded_sig_and_args);
    data.extend_from_slice(self.policy_id.as_bytes());
    data.extend_from_slice(&self.expiration.to_le_bytes());

    hash(&data).to_bytes()
}
```

Mitigation Suggestion

Hash encoded_sig_and_args and policy_id separately before adding it to the hash input

Resolution

Fixed in https://github.com/predicatelabs/sol-contracts/pull/18

[ND-PRED-IN-01] client_program isn't verified in UpdatePolicyId

Severity	Impact	Affected Component	Status
INFORMATIONAL	Event emitting wrong information	PolicyId	Fixed

The client_program parameter is never checked in UpdatePolicyId. In this case this doesn't result in invalid on-chain state, but the parameter is still used in the event that is being emitted.

Mitigation Suggestion

Remove the client_program parameter and instead read the pubkey of the corresponding account directly.

Resolution

Fixed in https://github.com/predicatelabs/sol-contracts/pull/21

[ND-PRED-IN-02] No domain separation for signed message

Severity	Impact	Affected Component	Status
INFORMATIONAL	Potential footgun if Attester key is reused	Attestation	Fixed

The message that is signed by the attester has no domain separation. The attester simply signs some arbitrary 32-byte hash. Right now, this key isn't used for anything else, but this pattern prevents footguns.

Relevant Code

```rust
/// Hash the statement for signature verification (equivalent to
hashStatementSafe in Solidity)
pub fn hash_statement_safe(&self) -> [u8; 32] {
    use anchor_lang::solana_program::hash::hash;

    let mut data = Vec::new();
    data.extend_from_slice(&self.uuid);
    data.extend_from_slice(&self.msg_sender.to_bytes());
    data.extend_from_slice(&self.target.to_bytes());
    data.extend_from_slice(&self.msg_value.to_le_bytes());
    data.extend_from_slice(&self.encoded_sig_and_args);
    data.extend_from_slice(self.policy_id.as_bytes());
    data.extend_from_slice(&self.expiration.to_le_bytes());

    hash(&data).to_bytes()
}
```

Mitigation Suggestion

Add a `b"predicate_solana_attestation"` (or without the solana if the same attestation should be used on different chains) to the front of the signed message.

Resolution

Fixed in https://github.com/predicatelabs/sol-contracts/pull/20

A | About Neodyme

Security is difficult.

To understand and break complex things, you need a certain type of people. People who thrive in complexity, who love to play around with code, and who don't stop exploring until they fully understand every aspect of it. That's us.

Our team never outsources audits. Having found over 80 High or Critical bugs in Solana's core code itself, we believe that Neodyme hosts the most qualified auditors for Solana programs. We've also found and disclosed critical vulnerabilities in many of Solana's top projects and have responsibly disclosed issues that could have resulted in the theft of over $10B in TVL on the Solana blockchain.

All of our team members have a background in competitive hacking. During such hacking competitions, called CTFs, we competed and collaborated while finding vulnerabilities, breaking encryption, reverse engineering complicated algorithms, and much more. Through the years, many of our team members have won national and international hacking competitions, and keep ranking highly among some of the hardest CTF events worldwide. In 2020, some of our members started experimenting with validators and became active members in the early Solana community. With the prospect of an interesting technical challenge and bug bounties, they quickly encouraged others from our CTF team to look for security issues in Solana. The result was so successful that after reporting several bugs, in 2021, the Solana Foundation contracted us for source code auditing. As a result, Neodyme was born.



B | Methodology

We are not checklist auditors.

In fact, we pride ourselves on that. We adapt our approach to each audit, investing considerable time into understanding the program upfront and exploring its expected behavior, edge cases, invariants, and ways in which the latter could be violated.

We use our uniquely deep knowledge of Solana internals, and our years-long experience in auditing Solana programs to find bugs that others miss. We often extend our audit to cover off-chain components in order to see how users could be tricked or the contract affected by bugs in those components.

Nonetheless, we also have a list of common vulnerability classes, which we always exhaustively look for. We provide a sample of this list here.

Select Common Vulnerabilities

Our most common findings are still specific to Solana itself. Among these are vulnerabilities such as the ones listed below:

- Insufficient validation, such as:
  - Missing ownership checks
  - Missing signer checks
  - Signed invocation of unverified programs
  - Account confusions
  - Missing freeze authority checks
  - Insufficient SPL account verification
  - Dangerous user-controlled bumps
  - Insufficient Anchor account linkage
- Account reinitialization vulnerabilities
- Account creation DoS
- Redeployment with cross-instance confusion
- Missing rent exemption assertion
- Casting truncation
- Arithmetic over- or underflows
- Numerical precision and rounding errors
- Anchor pitfalls, such as accounts not being reloaded
- Non-unique seeds
- Issues arising from CPI recursion
- Log truncation vulnerabilities
- Vulnerabilities specific to integration of Token Extensions, for example unexpected external token hook calls

Apart from such Solana-specific findings, some of the most common vulnerabilities relate to the general logical structure of the contract. Specifically, such findings may be:

- Errors in business logic
- Mismatches between contract logic and project specifications
- General denial-of-service attacks
- Sybil attacks
- Incorrect usage of on-chain randomness
- Contract-specific low-level vulnerabilities, such as incorrect account memory management
- Vulnerability to economic attacks
- Allowing front-running or sandwiching attacks

Miscellaneous other findings are also routinely checked for, among them:

- Unsafe design decisions that might lead to vulnerabilities being introduced in the future
    - Additionally, any findings related to code consistency and cleanliness
- Rug pull mechanisms or hidden backdoors

Often, we also examine the authority structure of a contract, investigating their security as well as the impact on contract operations should they be compromised.

Over the years, we have found hundreds of high and critical severity findings, many of which are highly nontrivial and do not fall into the strict categories above. This is why our approach has always gone way beyond simply checking for common vulnerabilities. We believe that the only way to truly secure a program is a deep and tailored exploration that covers all aspects of a program, from small low-level bugs to complex logical vulnerabilities.

C | Vulnerability Severity Rating

We use the following guideline to classify the severity of vulnerabilities. Note that we assess each vulnerability on an individual basis and may deviate from these guidelines in cases where it is well-founded. In such cases, we always provide an explanation.

Severity	Description
CRITICAL	Vulnerabilities that will likely cause loss of funds. An attacker can trigger them with little or no preparation, or they are expected to happen accidentally. Effects are difficult to undo after they are detected.
HIGH	Bugs that can be used to set up loss of funds in a more limited capacity, or to render the contract unusable.
MEDIUM	Bugs that do not cause direct loss of funds but that may lead to other exploitable mechanisms, or that could be exploited to render the contract partially unusable.
LOW	Bugs that do not have a significant immediate impact and could be fixed easily after detection.
INFORMATIONAL	Bugs or inconsistencies that have little to no security impact, but are still noteworthy.

Additionally, we often provide the client with a list of nit-picks, i.e. findings whose severity lies below Informational. In general, these findings are not part of the report.

Neodyme AG

Dirnismaning 55
Halle 13
85748 Garching
Germany

E-Mail: contact@neodyme.io

https://neodyme.io